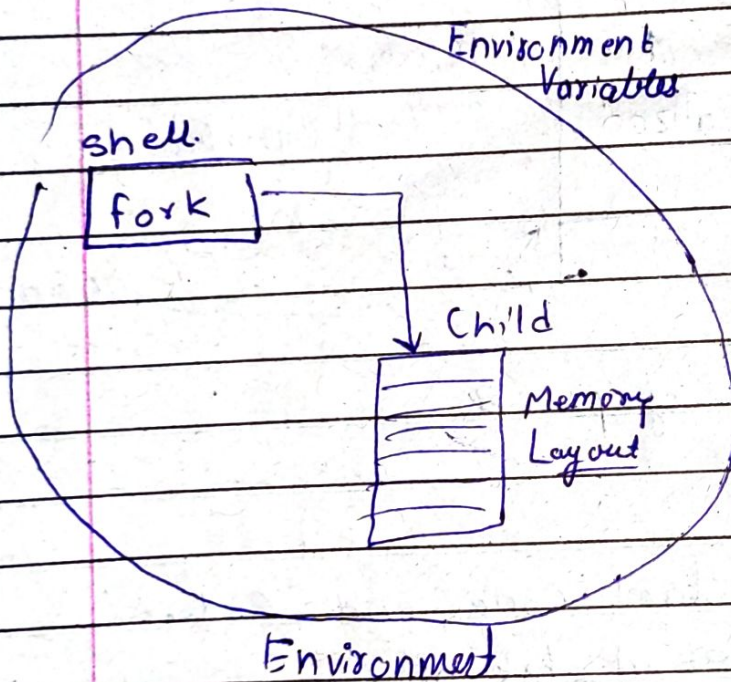


Sbrk $\rightarrow$ library function

119 Ammoniumchlorid

* Environment of Unix Process:

linker & loader. Put process in memory



MOV

ADD

MULT

SUB

JMP

JE

JNC

Who creates
Shell

init(1)
(first process)

- 1) Variables known to shell
- 2) getenv()
- 3) putenv()

4) export & Command


```
int main()
```

```
{
```

```
    signal(INTSIGINT, myfn);
```

```
    for(i=0; i < 5000; i++) {
```

```
        printf("I am in loop\n");
```

```
        Sleep(2);
```

```
    }
```

```
    return 0;
```

```
}
```

```
void myfn() {
```

```
    printf("Control C was Pressed\n");
```

```
}
```

I am in loop

" " " "

" " " "

^C (Control C was pressed

" " " "

" " " "

" " " "

Ctrl C → Default action → Terminate,

→ execute myfn(handler) on Signal (SIGINT)

→ SIGIGN → Ignore Signal.

What is name of terminal on android

SIGALRM - alarm()

SIGQUIT - ^\ (ctrl backslash)

SIGABRT - abort()

↳ Default action → abort (terminate process prematurely).

SIGURG - Urgent.

SIGPIPE

SIGCLD - When a child terminates

CHLD.

SIGABRT

SIGALARM

SIGBUS → implement

(43BSD) SIGCHLD } When a child
SYSV (SIGCLD } terminates / Stops.

↓
Parent.

SIGCONT → Continue.

SIGEMT → Emulation Trap

SIGFPE → Floating point Error / Out of Range.

SIGBUS

SIGINT → ^C ^break DEL.

SIGIO → Some events Occur in IO

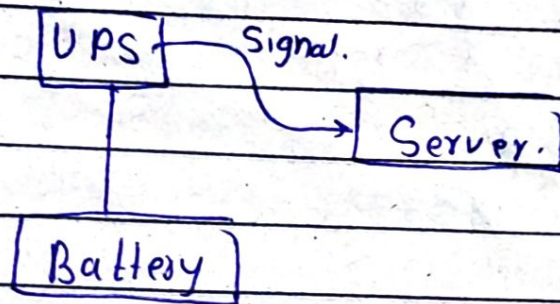
SIGKILL → Cant be Caught, Ignored to terminate the process

SIGPIPE → When 1 end is closed for communication.

SIGPOLL → From a pollable device.

SIGPROF → Profile Set by Settimer.

SIGPWR →



SIGQUIT → ^\ terminate process and generates Core

SIGSEGV - Segment Violated

Trying to access prohibited area of some/other process.

SIGSTOP

SIGSYS - Invalid system call.

SIGTERM → Terminating process.

SIGTRAP

SIGTSTP → ^Z Terminal generated.

SIGTTIN, SIGTTOU → Background Process.

Background process

↳ Does not take I/P from keyboard & doesn't display O/P.

Foreground is reverse of that one.

SIGURG → Used in TCP/IP.

SIGUSR1 } Signals provided
USR2 } doesn't.

SIGVTALRM SIGWINCH SIGXCPU
XFSZ.

Terminal
Keyboard display $\rightarrow$ Window.

#include <stdio.h>

#include <signal.h>

float $\rightarrow$ int

main()

{

printf ("One\n");

signal (SIGALRM, alarm_catch); // defines handler and sets the alarm

alarm(10);

pause();

printf ("two\n");

}

alarm_catch()

{

return;

}

Output : One

two

//after 10 seconds.

- We can implement functionality like Sleep() using SIGALRM alarm().


```
#include <Stdio.h>
```

```
#include <Stdlib.h>
```

```
#include <Signal.h>
```

```
- void Segfault(int dummy){
    printf("Segfault Occured");
    exit(1);
}
```

```
int main(){
    int *p = 0;
    signal(SIGSEGV, Segfault); //Signal Handler
    *p = 17;
    return 0;
}
```

Output: Segfault Occured

← Reason:

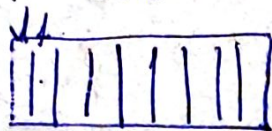
P is pointing to address 0

This address is virtual address

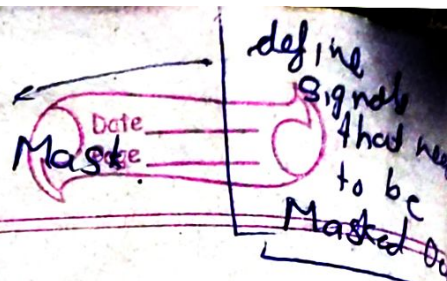
We are trying to put 17 at 0 address
but it does not belong to this process

∴ Segfault occurs.

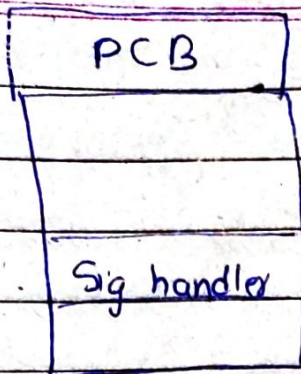
Mask



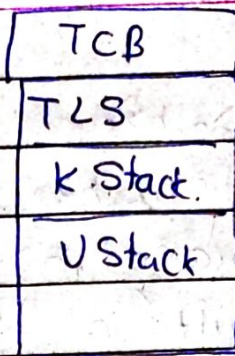
del



PCB



TCS



Process

Thread

Signal $\Rightarrow$ Shared by both process and handler thread.

```
#include <signal.h>
```

```
int sigaddset (sigset_t *set, int signo);
```

```
sigdelset (_____ "_____");
```

```
sigemptyset (_____ "_____");
```

```
sigfillset (_____ "_____");
```

```
sigismember (const sigset_t *set, int no)
```

```
pthread_sigmask()
```

```
int sigprocmask (int how, sigset_t *newmask, sigset_t *oldmask);
```

Sig action ();

System Call.

, 0 0 0

Date _____
Page _____

```
#include <Signal.h>
```

```
int main () {
```

```
    Sigset_t newmask, oldmask, pendmask;
```

```
    Signal (SIGQUIT, Sig_quit);
```

```
    Sigemptyset (& newmask);
```

```
    Sigaddset (& newmask, SIGQUIT);
```

```
    Sigprocmask (SIG_BLOCK, & newmask, & oldmask);
```

```
    Sleep (10);
```

```
    Sigpending (& pendmask);
```

```
    if (Sigismember (———, SIGQUIT))
```

```
        printf ("SigQuit Pending\n");
```

```
    Sigprocmask (SIG_SETMASK, & oldmask, NULL);
```

```
    printf ("SigQuit Unblocked\n");
```

```
    Sleep (10);
```

```
    exit (0);
```

```
}
```

```
Static void Sig_quit (int Sig no)
```

```
{ printf ("Caught SIGQUIT\n")
```

```
    Signal ((SIGQUIT, SIG_DFL);
```

```
#include <Stdio.h>
```

```
#include <Sys/user.h>
```

```
#include <Sys/ptrace.h>
```

```
int main() {
```

```
    int pid = fork ()
```

```
    if (pid == 0) {
```

```
        ptrace (PTRACE_TRACEME, 0, 0, 0);
```

```
        exec (" /bin/ls", "ls", 0);
```

```
    }
```



```

while(1){
    int status, struct user_regs_struct reg;
    waitpid(pid, &status, 0);
    if (WIFSTOPPED(status))
        printf("(child has been Stopped\n");
    ptrace (PTRACE_GETREGS, pid, 0, &reg);
    printf("eip = %lx esp = %lx", reg.eip, reg.esp);
    Sleep(10);
}

ptrace (PTRACE_SINGLESTEP, pid, 0, 0);
return 0;
}

```

- We trace out the execution of ls.
- We can see different contents of registers.

° Parent → Tracks. (Runs in while loop)
Child

- Parent waits for child using waitpid. When child runs 1 instruction it stops and send signal to parent.
- The parent then receives the signal and prints the contents of registers.

- ptrace Single Step tells child to execute one instruction and stop and send signal to parent.

- Child process is the process that we need to debug.

* Ptrace (request, pid, address, data);

PTRACE_ME - TRACE ME

PTRACE - ATTACH

PTRACE - DETACH

PTRACE - PEEKTEXT

PTRACE - PEEKDATA

PTRACE - USER

PTRACE - POKE TEXT

PTRACE - POKE DATA

PTRACE - POKE USER

PTRACE - SYSCALL

↳ strace calls

PTRACE - CONT

PTRACE - SINGLESTEP

PTRACE - KILL

PTRACE - GET REGS

PTRACE - GET FP REGS

PTRACE - GET FPU REGS

PTRACE SETREGS

PTRACE - SETFPREGS

PTRACE SETFPXREGS

debugger is a program which got statement
ptrace.

Attach → If we want to debug a running process we attach to that process using its pid.

PEEKTEXT → To find expressions (at a particular location)

PEEK DATA → To find variable its data.

POKE → Modify.

POKE DATA → Modify data area

POKEUSER → Modify user area.

PTRACE CONT -

PTRACE_KILL -